Lecture 11

RISC-V: Handling Multicycle Operations in Pipeline

Instructor: Dr. Muhammad Imran

CHIPXPRT

NCDC
NUST CHIP DESIGN CENTRE

Acknowledgement

- Content from following has been used in these lectures
  - Computer Organization and Design, RISC-V 2nd Edition, Patterson and Hennessy

Contents

- Multicycle Operations

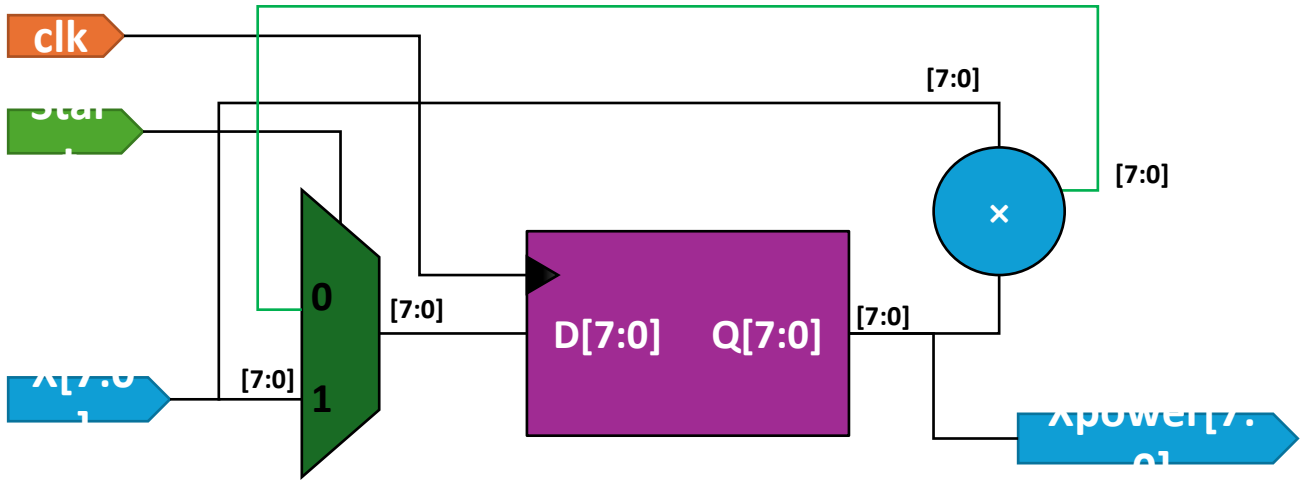
Multicycle Operations

Why multicycle?

- What happens if we force all operations to complete in one cycle?
  - Slower clock
  - More logic
  - Can be both!
- Pipelining multicycle operation
  - Start a new operation when the first operation goes into the second stage of execution!
    - For example, pipeline a multiplier into seven pipeline stages
      - Can overlap execution of 7 (independent) multiply operations!
- Can we fully pipeline multicycle operations?
  - Sometimes no!

Unpipelined Multicycle Execution

```
Xpower = 1;
for(i=0; i < 3; i++)
        Xpower =
X*Xpower;
```



- This design can, however, be fully pipelined!
  - At cost of additional registers and multipliers!

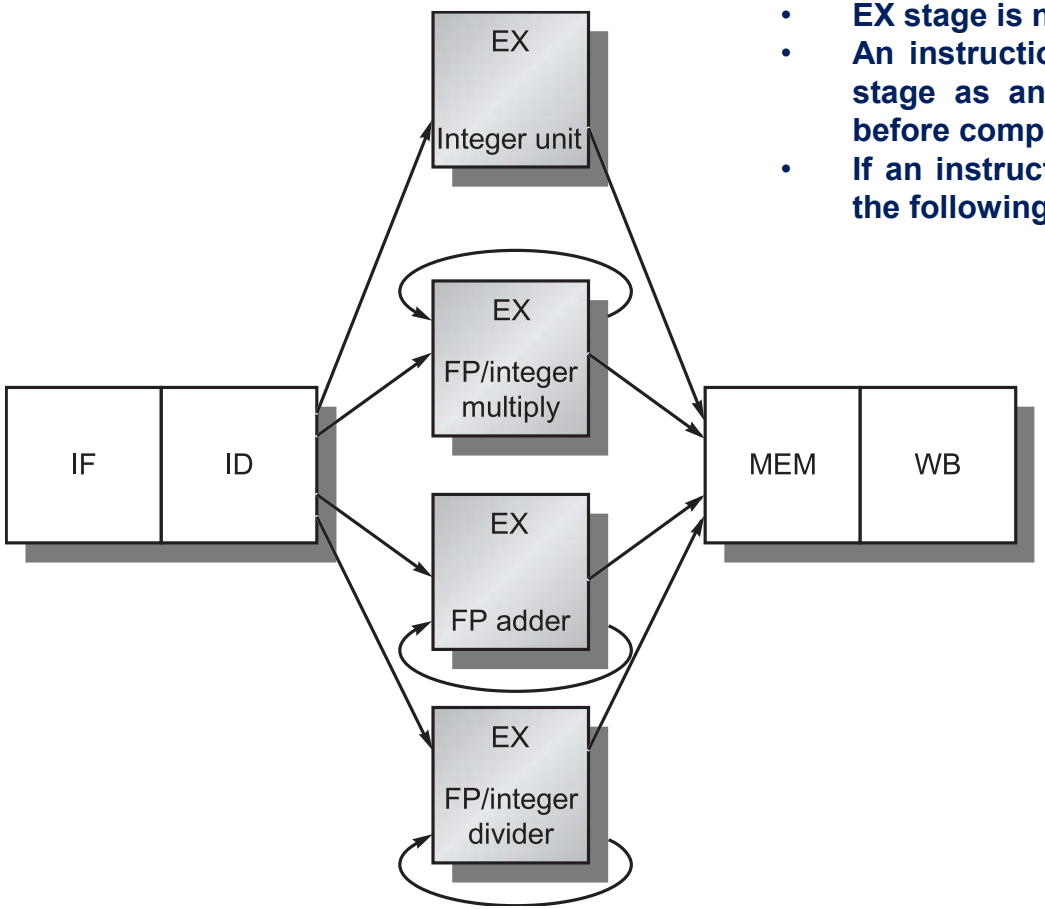
We will see how to handle both pipelined and unpipelined multicycle operations …

RISC-V Floating Point Pipeline

- Change the EX stage
  - Allow operations to take multiple cycles for different operations!
- Add multiple functional units instead of one ALU
  - Benefit?
    - Reduced structural hazards!
      - Cannot issue a second instruction to EX stage, if the functional unit is busy!
- Functional units in FP pipeline
  - Integer unit for loads, stores, integer ALU operations, branches!
  - FP and integer multiplier
  - FP adder that handles FP add, subtract, and conversion
  - FP and integer divider

RISC-V Floating Point Pipeline

- FP pipeline with unpipelined functional units



- **EX stage is not pipelined but it is multicycle!**
- **An instruction using same functional unit in EX stage as an earlier instruction cannot proceed before completion of previous one!**
- **If an instruction cannot proceed to EX stage, all the following instructions are stalled!**

RISC-V Floating Point Pipeline

- Pipelining the functional units
  - Let's say
    - FP/Integer multiply unit is pipelined into 7 stages!
    - FP adder is pipelined into 4 stages!
    - FP/Integer divide unit is **unpipelined** with 25 stages!
- Use latency
  - No of cycles between an instruction producing a result and an instruction using that result!
    - After how many cycles dependent instruction can start execution!
      - Just like loads always have use latency (wait) of 1 cycle!
- Initiation Interval
  - When can another instruction that uses the same functional unit, be issued (assuming it is independent)!

RISC-V Floating Point Pipeline

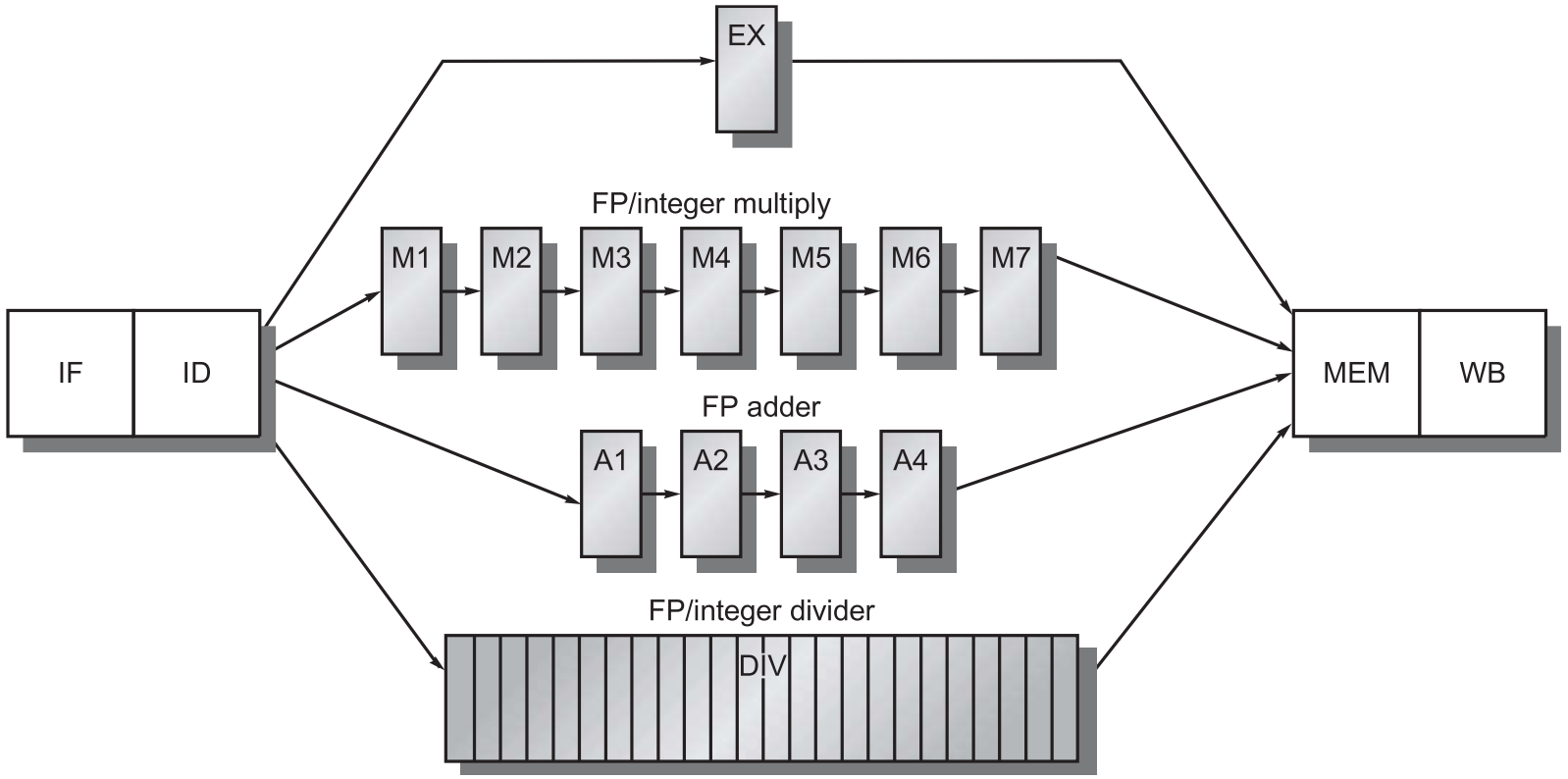
- Latencies and initiation intervals of functional units

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

- These are use latencies of instruction using the results in EX stage!
  - What about use latencies for store instruction for the value being stored (not the base address register)?
    - Can be one cycle less!
    - Because the stored value is needed in MEM stage not EX stage!

RISC-V Floating Point Pipeline

- FP pipeline with pipelined functional units



- **DIV is still not pipelined!**
- **We can have up to 7 multiply, 4 FP add and 1 divide operation in the pipeline at same time!**

RISC-V Floating Point Pipeline

- Penalty for faster clock and more pipeline stages in a functional unit?
  - More latency!
    - Can cause stalls due to data hazards!
- Additional Pipeline Registers
  - ID register can be logically divided into multiple registers, one each for every functional unit
    - ID/EX, ID/A1, ID/M1, and ID/DIV
    - Can also be implemented using different reigsters
    - However, there's only one operation that can be in this stage at a time and control signals are associated only with that operation!
      - That is in-order instruction issue!

RISC-V Floating Point Pipeline

- Executing a sequence of independent instructions
  - fmul # multiply
  - fadd # add
  - flw # floating point load
  - fsw # floating point store

fmul	IF	ID	M1	M2	M3	M4	M5	M6	**M7**	MEM	WB
fadd		IF	ID	A1	A2	A3	**A4**	MEM	WB		
flw			IF	ID	EX	**MEM**	WB				
fsw				IF	ID	EX	**MEM**	WB			

Hazards and forwarding is handled in a similar manner as integer pipeline, with a few additional issues …

Forwarding is very similar, only hazard detection unit need to consider a few more situations for adding stalls…

Hazards in FP Pipeline

- Structural Hazards
  - Unpipelined divide unit
  - Instructions using MEM, WB at same time!
- Instruction have varying running times
  - Can have more than one register writes in same cycle!
- Write after write (WAW) hazards are possible
  - A later instruction writes the result to a register before an earlier one!
    - When the earlier one finishes, the result can be wrong!
- Write-after-Read (WAR) hazard still not possible !
  - Because instructions enter execution in-order!

Hazards in FP Pipeline

- Instructions complete / commit in an order different than the issuance order
  - Exceptions' handling is complex!
- Because of longer EX stage
  - Stalls after RAW hazards would be more frequent!
  - RAW: Instruction needing data yet to be produced by earlier one!

Hazards in FP Pipeline

- Read after write (RAW) hazards
  - Consider following code sequence
    - fld f4, 0(x2)
    - Fmul.d f0, f4, f6
    - fadd.d f2, f0, f8
    - fsd f2, 0(x2)
  - Execution in the pipeline

Is forwarding being used?

	Clock cycle number																
Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
`fld f4,0(x2)`	IF	ID	EX	MEM	WB												
`fmul.d f0,f4,f6`		IF	ID	Stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
`fadd.d f2,f0,f8`			IF	Stall	ID	Stall	Stall	Stall	Stall	Stall	Stall	A1	A2	A3	A4	MEM	WB
`fsd f2,0(x2)`					IF	Stall	Stall	Stall	Stall	Stall	Stall	ID	EX	Stall	Stall	Stall	MEM

Unless the previous instruction is issued for execution, the next instruction is not decoded!

Because control signals in ID/EX register are for one instruction and we are not replicating hardware!

Extra Stall to avoid conflict of MEM stages in two instructions!

Hazards in FP Pipeline

- Structural hazard due to multiple write backs!
  - Consider following sequence!

	Clock cycle number										
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**
`fmul.d f0,f4,f6`	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
`fadd.d f2,f4,f6`				IF	ID	A1	A2	A3	A4	MEM	WB
...					IF	ID	EX	MEM	WB		
...						IF	ID	EX	MEM	WB	
`fld f2,0(x2)`							IF	ID	EX	MEM	WB

  - In 10th cycle only fld accesses memory, so there's no structural hazard!
  - But in 11th cycle, all three instructions are in WB stage?
    - What should be done, assuming single write port for register file?

Hazards in FP Pipeline

- Structural hazard due to multiple write backs!
  - Solutions
    - Add multiple ports
      - Since this case is rare, it is not worth to add more ports!
    - Detect it as a structural hazard and then add a stall to resolve it!
      - Two ways to detect and add stall
        1. Track the use of write port in ID stage and stall the instruction!
           - A shift register can indicate when an instruction will use write port!
           - Benefits?
             - All stalls are detected in ID stage as for other stalls!
           - Cost is additional shift register / write conflict logic!
        2. Stall an instruction when it tries to enter MEM or WB stage
           - In this case, any of the conflicting instructions can be stalled!
           - Give priority to the unit with longest latency as that would have most probably caused others to stall because of a RAW hazard, eventually leading to both instructions writing back at same time!
           - Benefit is easier to detect and implement stall here!
           - Drawback is complicated control as this stall will impact multiple instructions!

Hazards in FP Pipeline

- Write after write (WAW) hazards!
  - Two instructions write same register but in wrong order!
  - Consider the following case!

fadd f2, f4, f6	IF	ID	A1	A2	A3	A4	MEM	WB
		IF	ID	EX	MEM	WB		
flw f2, 0(x2)			IF	ID	EX	MEM	WB	

  - WAW hazard only occurs when the result of an instruction is overwritten without usage by another instruction!
    - If another instruction was using result of fadd i.e., f2, then there would be RAW hazard and fld will be delayed automatically!

Hazards in FP Pipeline

- Write after write (WAW) hazards!
  - Solutions
    1. Delay issuance of second instruction until the first one enters MEM stage!
    2. Allow the second instruction to proceed but do not allow the previous instruction to write the register again!
       - Since the result is not being used, no need to write it!
    - Both can be implemented by detecting WAW hazard in ID stage!
    - The difficulty is to detect when a second instruction may finish before an earlier one!
    - Since this case is rare, a simple solution can be used
      - Do not issue an instruction which writes back the same register as an earlier one which is in execution!

Summing up the hazard detection in FP pipeline …

Hazards in FP Pipeline

- Detecting hazards
  - Between integer instructions!
    - Only for load use or branches if branch is decided earlier!
  - Between FP instructions
  - Between FP and integer instructions
    - For FP loads or stores!
    - For data movement between FP and integer registers!

Hazards in FP Pipeline

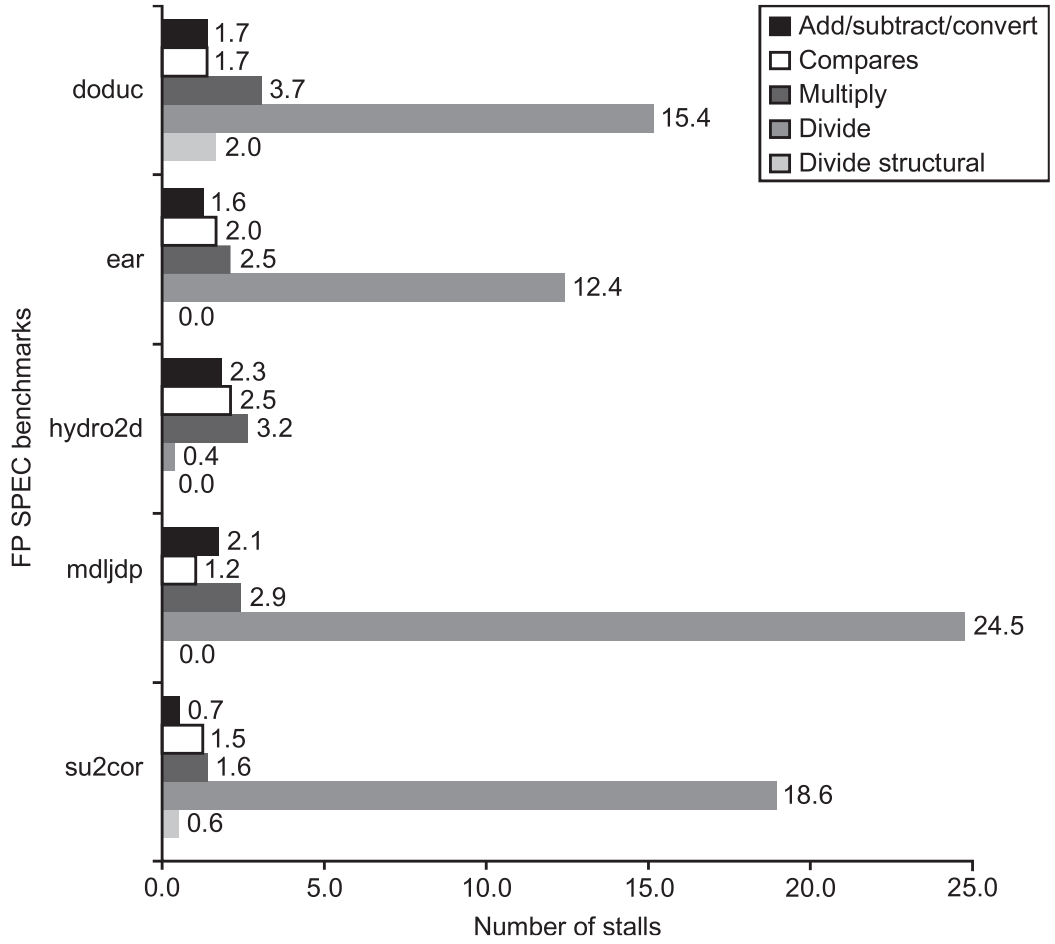
- Assuming that all hazards are detected in ID stage, 3 checks need to be performed
  1. Check for structural hazards!
     - Check if divide unit is busy or not!
     - Check if write port will be available when needed!
  2. Check for RAW hazards!
     - Source registers should not be pending destinations in the pipeline when this instruction will need them in pipeline!
     - A number of checks need to be made for this!
  3. Check for a WAW hazard!
     - If an instruction in A1, ..., A4, D, M1, ..., M7 has same destination as this one,
       - Stall this instruction!

Hazards in FP Pipeline

- Forwarding is similar as in integer pipeline!
  - Test for forwarding
    - Check if destination in EX/MEM, A4/MEM, M7/MEM, D/MEM and WB/MEM pipeline register is same as the source operands of current instruction (in execution)!

Stalls Frequency in FP Pipeline

- How efficient a FP pipeline performs?



Thank You